
Terraform Variables

1. What Are Variables in Terraform?

Terraform **variables** are used to make your configuration **dynamic, flexible, and reusable**.

They allow you to:

- Avoid hardcoding values
- Reuse the same Terraform code across different environments
- Pass values from `terraform.tfvars`, environment variables, CLI, or modules
- Create cleaner and more maintainable infrastructure code

Variables make Terraform behave like a real programming language — you write the logic once and reuse it with different inputs.

2. Why Do We Use Variables?

✓ Flexibility

You can change region, VPC CIDR, subnet CIDRs, and tags without touching `main.tf`.

✓ Reusability

The same Terraform code works for **dev, stage, prod, or training environments**.

✓ Security

Sensitive values (like access keys) can be stored confidentially using variable types and tfvars.

✓ Clean Code

Your main.tf file becomes shorter, readable, and professional.

✓ Industry Standard

All companies follow “variables + tfvars per environment” structure.

🧩 3. How Variables Work in Terraform

Variables have 3 parts:

1 Definition — inside **variables.tf**

```
variable "region" {  
  type    = string  
  default = "us-east-2"  
}
```

2 Usage — inside **main.tf**

```
region = var.region
```

3 Assign values (optional) — inside **terraform.tfvars**

```
region = "us-east-2"
```

Terraform automatically picks values in the following order:

1. CLI `-var`
 2. `terraform.tfvars`
 3. Default values in `variables.tf`
-

4. Types of Variables

Terraform supports different variable types:

✓ **string**

"hello"

✓ **number**

10

✓ **bool**

true

✓ **list**

["subnet-1", "subnet-2"]

✓ **map**

{ key1 = "value1", key2 = "value2" }

✓ **object**

{

```
name = string
cidr = string
}
```

In your project, most variables are **string**.

5. Variables Used in Your VPC Project (with explanations)

Below is the complete list of variables you are using with **clear explanation for each one**.

1. region

```
variable "region" {
  type    = string
  default = "us-east-2"
}
```

Purpose:

Specifies the AWS region where Terraform will deploy your infrastructure.

Example:

- Ohio → **us-east-2**
 - Virginia → **us-east-1**
-

2. project

```
variable "project" {  
  type    = string  
  default = "complete"  
}
```

Purpose:

Used as a **prefix** for naming AWS resources and applying tags.

Example tags generated:

- complete-vpc
- complete-public-subnet
- complete-private-rt

Helps organize resources belonging to the same project.

3. vpc_cidr

```
variable "vpc_cidr" {  
  type    = string  
  default = "10.0.0.0/16"  
}
```

Purpose:

Defines the VPC IP range.

A **/16** means:

- 65,536 IPs
 - Suitable for large architectures
-

◆ 4. public_subnet_cidr

```
variable "public_subnet_cidr" {  
  type    = string  
  default = "10.0.1.0/24"  
}
```

■ Purpose:

IP range for the **public subnet** where resources get a public IP.

◆ 5. private_subnet_cidr

```
variable "private_subnet_cidr" {  
  type    = string  
  default = "10.0.2.0/24"  
}
```

■ Purpose:

IP range for the **private subnet** where resources do NOT get public IPs.

◆ 6. availability_zone

```
variable "availability_zone" {  
  type    = string
```

```
default = "us-east-2a"
}
```

Purpose:

Defines which AWS Availability Zone to place subnets in.

Examples:

- us-east-2a
- us-east-2b
- us-east-2c

This enables multi-AZ architecture.

7. Access key & secret key (if used temporarily)

In learning environments only (NEVER production):

```
variable "access_key" {
  type    = string
  sensitive = true
}
```

```
variable "secret_key" {
  type    = string
  sensitive = true
}
```

Terraform marks these as **sensitive**, so they won't show in logs.

But best practice:

- ❗ Never store them inside code
 - ✓ Store in environment variables or AWS CLI config
-

🧠 6. How These Variables Work Together in Your VPC

Example from your `main.tf`:

Using region:

```
provider "aws" {  
  region = var.region  
}
```

Using project:

```
tags = merge(local.common_tags, { Name = "${var.project}-vpc" })
```

Using subnet cidrs:

```
cidr_block = var.public_subnet_cidr
```

Using availability zone:

```
availability_zone = var.availability_zone
```

This makes your VPC code **fully dynamic**.

7. Why This Is Important for Students

- ✓ How real DevOps teams write Terraform
 - ✓ How to avoid duplication
 - ✓ How to scale infra without rewriting code
 - ✓ How to work with different environments
 - ✓ How to build **production-ready infrastructure**
-

8. Final Summary

Terraform Variables:

- Make configuration reusable
- Improve maintainability
- Avoid hardcoding
- Allow environment-specific customization
- Enable cleaner & modular designs

In your VPC:

You used variables for:

- Region
- Naming
- VPC CIDR
- Subnet CIDRs

- AZ selection
- AWS credentials (temporary)

This is exactly how Terraform is taught in professional DevOps training.
